INTERNSHIP REPORT

ON

DATA ANALYSIS

A Report Submitted in partial fulfilment of

the requirements for the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

Submitted by,

M. Sai Ratna Vaishnavi roll no:160622737101

J. Sruthi roll no:160622737092

P. Shravya roll.no:160622737111

Under the guidance of

Dr. KURRA.MALLAIAH

Scientist, 'F', DRDL, Hyderabad

(Duration: 12th November,2024 to 27th January,2025)

STANLEY COLLEGE OF ENGINEE RING AND TECHNOLOGY FOR WOMEN(AUTONOMOUS)

Chapel Road, Abids, Hyderabad –500 001

(Affiliated to Osmania University & Approved by AICTE)

(All eligible UG Courses are accredited by NBA & Accredited by NAAC with 'A' Grade)

Department of Information Technology

[2024-2025]

STANLEY COLLEGE OF ENGINEERING AND TECHNOLOGY FOR WOMEN(AUTONOMOUS)

(Affiliated to Osmania University & Approved by AICTE) (All eligible UG Courses are accredited by NBA & Accredited by NAAC with 'A' Grade) Chapel Road, Abids, Hyderabad – 500 001

Department Of Information and Technology

CERTIFICATE

This is to certify that the "Internship report on Data Analysis" submitted by "M. Sai Ratna Vaishnavi-(160622737101), J. Sruthi-(160622737092) and P. Shravya-(160622737111)" is work done by them and submitted during 2024– 2025 academic year, in partial fulfilment of the requirements for the award of the degree of Bachelor Of Engineering in Information Technology, at DRDL, Hyderabad.

Internship Supervisor

Dr. K. Mallaiah

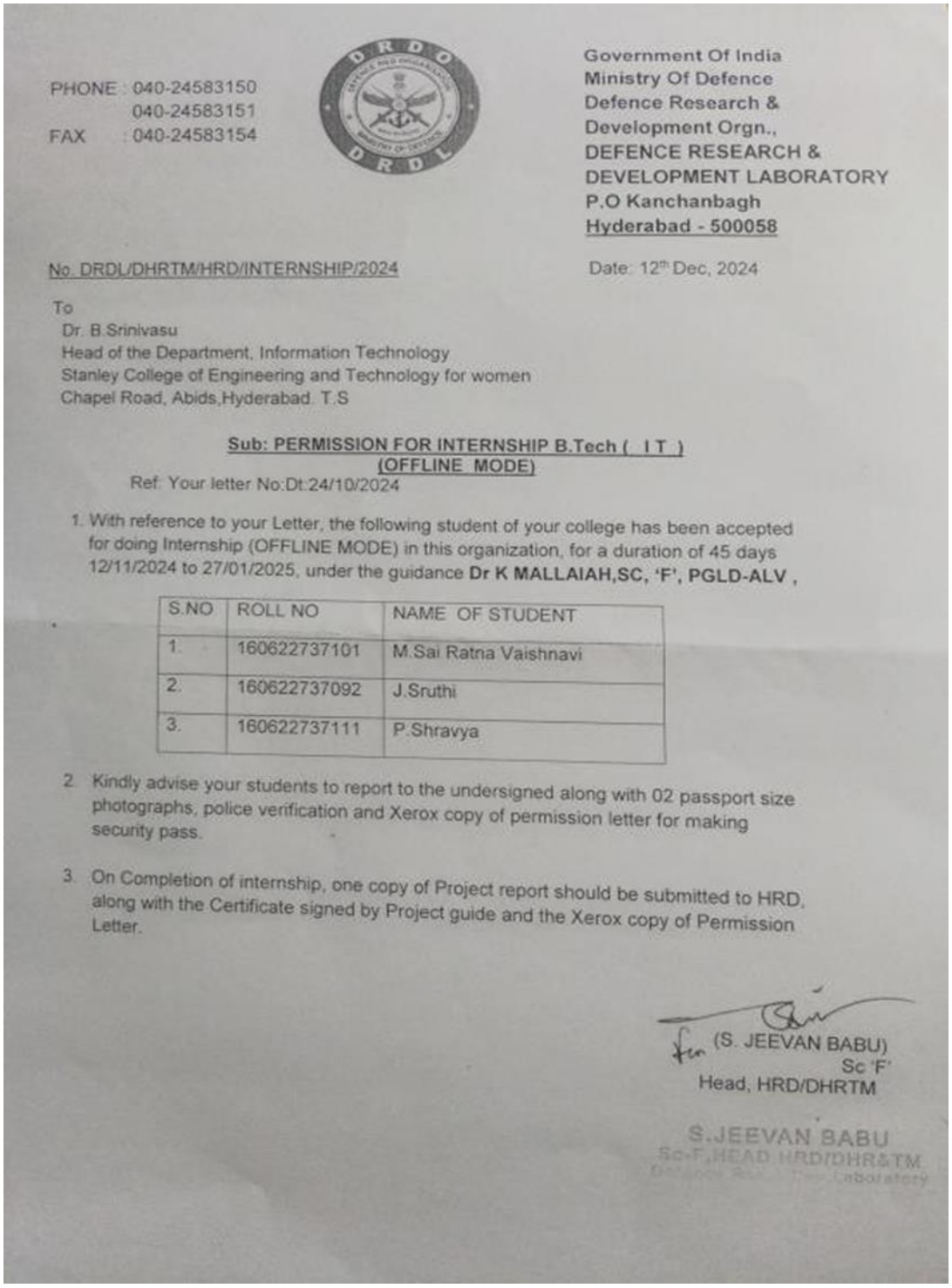
Scientist 'F'

DRDL, Hyderabad.

Head of the Department

Dr. B. Srinivasu

OFFER LETTER



PHONE : 040-24583150
040-24583151
FAX : 040-24583154

Government Of India
Ministry Of Defence
Defence Research &
Development Orgn.,
DEFENCE RESEARCH &
DEVELOPMENT LABORATORY
P.O Kanchanbagh
Hyderabad - 500058

No. DRDL/DHRTM/HRD/INTERNSHIP/2024

Date: 12th Dec, 2024

To
Dr. B.Srinivasu
Head of the Department, Information Technology
Stanley College of Engineering and Technology for women
Chapel Road, Abids,Hyderabad. T.S

Sub: PERMISSION FOR INTERNSHIP B.Tech (I T)
(OFFLINE MODE)

Ref: Your letter No:Dt:24/10/2024

1. With reference to your Letter, the following student of your college has been accepted for doing Internship (OFFLINE MODE) in this organization, for a duration of 45 days 12/11/2024 to 27/01/2025, under the guidance **Dr K MALLAIAH,SC, 'F', PGLD-ALV ,**

S.NO	ROLL NO	NAME OF STUDENT
1.	160622737101	M.Sai Ratna Vaishnavi
2.	160622737092	J.Sruthi
3.	160622737111	P.Shravya

2. Kindly advise your students to report to the undersigned along with 02 passport size photographs, police verification and Xerox copy of permission letter for making security pass.

3. On Completion of internship, one copy of Project report should be submitted to HRD, along with the Certificate signed by Project guide and the Xerox copy of Permission Letter.

for (S. JEEVAN BABU)
Sc 'F'
Head, HRD/DHRTM

S.JEEVAN BABU
Sc-F,HEAD HRD/DHR&TM
Defence Research & Development Laboratory

ACKNOWLEDGEMENT

First, we would like to thank **Dr. Kurra. Mallaiah, Scientist 'F'** DRDL (Defence Research and Development Laboratory), DRDO (Defence Research and Development Organization), Govt.of India, Hyderabad, for giving us the opportunity to do an internship within the organization.

We are highly indebted to **Principal (Dr .Satya Prasad Lanka)**, for the facilities provided to accomplish this internship.

We would like to thank my **Head of the Department (Dr . B. Srinivasu)** for his constructive criticism throughout our internship.

We are extremely great full to my department staff members and friends who helped me in successful completion of this internship.

M. Sai Ratna Vaishnavi-(160622737101)

J. Sruthi-(160622737092)

P. Shravya-(160622737111).

ABSTRACT

Missile subsystem development involves extensive testing and data analysis to evaluate performance and optimize functionality. The development and testing of missile subsystems, including guidance, propulsion, radar processing, and onboard computing, generate large volumes of data that require efficient analysis for performance evaluation and optimization. This project aims to design and implement an interactive Graphical User Interface (GUI) using Tkinter to streamline the selection, loading, and visualization of missile subsystem test data.

The application enables users to select a specific missile type and its associated subsystems, upload CSV or Excel files containing test data, and dynamically choose relevant columns for visualization. Pandas is employed for data handling, ensuring efficient loading, extraction, and manipulation of datasets. Plotly is integrated to generate interactive line plots, allowing users to analyze trends by selecting an X-axis and multiple Y-axis variables. This feature enhances the ability to compare different parameters within a subsystem, aiding engineers and analysts in data-driven decision-making.

Additionally, Pillow is utilized for image handling, enabling the display of relevant images such as the DRDO logo and missile illustrations within the interface for professional and intuitive user experience. The combination of these technologies provides a comprehensive and user-friendly tool for the visualization and interpretation of missile subsystem performance data, contributing to improved system evaluation and development processes.

TABLE OF CONTENTS

S.NO.	CONTENT	PAGE NO.
	ACKNOWLEDGEMENT	i
	ABSTRACT	ii
1.	Introduction 1.1 Problem Statement 1.2 Problem Objective	1
2.	Background knowledge 2.1 About the organization 2.2 Project Description	2-5
3.	Objectives and Goals 3.1 Objectives 3.2 Goals	6-7
4.	Technology stack and Skills acquired 4.1 Technology stack 4.2 Skills acquired	8-9
5.	Achievements and Contributions 5.1 Achievements 5.2 Contributions	10-11
6.	Methodology 6.1 Install of Python 6.2 Install of Python Libraries	12
7.	Code	13-15
8.	Result Screens	16-18
9.	Conclusion	19

1.INTRODUCTION

1.1Problem Statement

In the field of defense research and missile system development, analyzing and visualizing test data from various missile subsystems is crucial for defence research, performance evaluation, optimization and development. Missile subsystems such as guidance, propulsion, and seeker mechanisms generate large volumes of data during testing, which need to be systematically processed to assess performance, detect anomalies, and validate system reliability.

Traditional methods of handling this data involve manually browsing through spreadsheets, which can be inefficient, error-prone, and time-consuming.

A user-friendly solution is needed to streamline this workflow, enabling quick insights through interactive plotting and visualization by creating a an intuitive tool that allows seamless selection of specific missile subsystems, easy loading of test datasets, and dynamic visualization of key parameters.

1.2 Problem Objective

The objective of this project is to develop an interactive graphical user interface (GUI) using Tkinter that allows users to select, load, and visualize missile subsystem data efficiently. The tool provides the following functionalities:

- **Subsystem Selection:** Users can select a specific missile type and its associated subsystems for analysis.
- **Data Loading:** Users can upload CSV or Excel files containing test data for the selected subsystem.
- **Dynamic Column Selection:** The application dynamically detects columns in the uploaded dataset, allowing users to choose X-axis and multiple Y-axis variables for visualization.
- **Interactive Data Visualization:** The application utilizes Plotly to generate interactive line plots, aiding in performance analysis and decision-making.
- **User-Friendly Navigation:** The GUI includes intuitive navigation with buttons, dropdowns, and scrollbars for easy selection and visualization.

2.BACKGROUND KNOWLEDGE

The development of missile systems involves rigorous testing and evaluation of various subsystems such as guidance, propulsion, radar processing, and onboard computing. Each of these subsystems generates large amounts of test data that need to be analyzed to assess performance, identify trends, and optimize system functionality. Python provides a combination of Graphical User Interface (GUI) development and data visualization libraries, making it a powerful choice for building interactive applications for better data handling and visualization

The key Technologies used are:

Tkinter (GUI Development)

- Tkinter is a built-in Python library for creating graphical user interfaces.
- It allows users to interact with the application using buttons, dropdowns, and file selection dialogs.
- The program utilizes Tkinter to create windows, display images, and handle user inputs for selecting missile subsystems and data files.

Pandas (Data Handling & Processing)

- Pandas is used for reading and processing test data stored in CSV or Excel files.
- It enables easy manipulation of datasets, including column selection and data extraction.
- The program dynamically loads the test data and provides users with selectable columns for visualization.

Plotly (Interactive Data Visualization)

- Plotly is a powerful Python library for creating interactive plots.
- It allows users to visualize missile subsystem test data through dynamic line graphs.
- The program enables users to select an X-axis and multiple Y-axes for plotting, making data comparison intuitive.

Pillow (Image Handling)

- Pillow is used to load and display images (such as the DRDO logo and missile images) in the GUI.
- The program resizes and places images within the Tkinter interface for a professional look.

2.1 About the Organization

Realizing the importance of guided missile weapon systems in the modern warfare, a Special Weapon Development Team (SWDT) was formed in 1958. This team was later expanded into **DRDL,** a full-fledged laboratory in June 1961 at the campus of Defence Science Centre, Delhi. The laboratory was moved to Hyderabad in February 1962, from where starts the story of guided missiles in India.

During the initial phase, the laboratory successfully developed an anti-tank missile system and indigenous rockets and proved them through flight trails. In 1972, Project Devil, for the development of a medium range Surface-to-Surface Missile was initiated. A large number of infrastructure and test facilities were established during the period.

DRDL is responsible for the design and development of state-of-the-art Missile Systems and technologies required for the deterrence and defence of the country. Systems and Technologies developed by DRDL are deployed on underwater, sea, land as well as air-based platforms.

DRDL is working on several technologies required for the Missiles and Strategic Systems including; Aerodynamics and Airframe Design, Computational Fluid Dynamics, Solid, Liquid, Ramjet and Scramjet Propulsion, Precision Fabrication, Systems Analysis, as well as the Command-and-Control systems for missile-based weapon systems.

DRDL is entrusted with the design, development and leading-to-production of systems and technologies for missile-based weapon systems deployable from underwater, sea, land and air-based platforms. It ensures Ensure product support, product upgrade and end-of-life replacement and establish critical infrastructure and facilities for design, testing and evaluation.

2.2 Project Description

The program provides an interactive GUI for selecting missile subsystems, loading test data, and visualizing it using interactive plots. Below is a step-by-step explanation of how the program works:

1. **Importing Required Libraries:**
    - The script imports Tkinter for GUI handling, Pandas for data processing, Plotly for visualization, and PIL for image handling.
2. **Defining Static Data:**
    - A dictionary contains predefined missile system categories and their associated subsystems.
3. **Creating GUI Components:**
    - The main Tkinter window is initialized with a title and a predefined size.
    - Labels, buttons, and images are added to enhance the user interface.
4. **Handling Button Clicks and File Selection:**
    - When a missile system button is clicked, a new window opens displaying subsystems.
    - Users can select a subsystem, triggering a file selection dialog for data input.
5. **Loading and Validating Data:**
    - The selected CSV or Excel file is read using Pandas.
    - If the file is empty or in an incorrect format, an error message is displayed.
6. **Creating a Plotting Window:**
    - A new Tkinter window is opened where users can choose X and Y axes for visualization.
    - Checkboxes allow multiple Y-axis selections.
7. **Generating Plots with Plotly:**
    - The selected columns are passed to Plotly's line plot function.
    - Legends and titles are dynamically updated for better readability.
8. **Navigation and Error Handling:**
    - Back buttons are placed to allow users to navigate between windows.
    - Exception handling is implemented for smoother operation.

3.OBJECTIVES AND GOALS

3.1 Objectives:

Develop a GUI-Based Data Visualization Tool

- Provide an interactive graphical interface using **Tkinter** to allow users to explore missile system components and their data.

Enable Data Selection and Analysis

- Allow users to select missile subsystems and load associated datasets in **CSV** or **Excel** formats.
- Provide dynamic column selection for data analysis.

Integrate Interactive Data Plotting

- Use **Plotly** to generate interactive **line charts** for selected datasets, allowing better visualization and understanding.

Enhance Usability with a Structured UI

- Display categorized missile subsystems with labeled buttons for easy navigation.
- Implement a file selection dialog for user convenience.

Ensure Data Handling and Error Management

- Validate file formats and handle missing or empty files to prevent runtime errors.
- Display appropriate error messages for invalid operations.

Improve User Experience with Visual Elements

- Display **DRDO** and missile-related images in the main window to provide a professional and domain-specific touch.
- Use **color-coded buttons** and structured layouts for an intuitive interface.

3.2 Goals:

User-Friendly Interaction

- Ensure the GUI is easy to navigate for researchers, analysts, and engineers working with missile subsystem data.

Efficient Data Processing

- Support **quick data loading** and selection to streamline the visualization process.

Scalability and Customization

- Allow the system to be expanded for more missile types and subsystems in the future.

High-Quality Data Visualization

- Use **interactive plots** with legends and labels to improve data interpretation.

Error-Free and Robust Performance

- Implement error handling mechanisms to **prevent crashes** and provide meaningful user feedback.

4.TECHNOLOGY STACK AND SKILLS ACQUIRED

4.1Technology Stack:

1.Programming Language:

- **Python** – Used for scripting and implementing the GUI, data processing, and visualization.

2.Graphical User Interface (GUI):

- **Tkinter** – A built-in Python library used for creating the interactive graphical interface.

3.Data Handling and Processing:

- **Pandas** – Used for reading, processing, and analyzing CSV/Excel files.

4.Data Visualization:

- **Plotly Express** – Enables the creation of interactive line plots for data representation.

5.File Handling:

- **tkinter.filedialog** – Used for selecting and opening data files dynamically.

6.Image Processing:

- **PIL (Pillow)** – Used for loading, resizing, and displaying images in the GUI.

7.Event-Driven Programming:

- **Tkinter Event Handling** – Handles button clicks, dropdown selections, and window events.

4.2Skills Acquired:

1.GUI Development in Python

- Designing interactive user interfaces using Tkinter.
- Structuring windows, frames, labels, buttons, and dropdowns dynamically.

2.Data Manipulation and Processing

- Reading CSV/Excel files using Pandas.
- Handling missing data and validating file formats.

3.Interactive Data Visualization

- Creating line charts with multiple Y-axis selections using Plotly.
- Customizing plot legends, titles, and axes.

4.File Handling and User Interaction

- Implementing file selection dialogs for dynamic user input.
- Handling exceptions to prevent crashes due to invalid file selections.

5.Event Handling and Function Binding

- Using Lambda functions to pass dynamic values in button callbacks.
- Managing Tkinter event loops for smooth user interaction.

6.Image Processing in GUI Applications

- Loading and displaying images using Pillow (PIL).
- Adjusting image sizes for better UI integration.

7.Error Handling and Debugging

- Implementing try-except blocks to handle file errors.
- Displaying meaningful error messages to the user.

5.ACHIEVEMENTS AND CONTRIBUTIONS

5.1 Achievements:

1.Developed a Functional GUI for Data Analysis

- Successfully created an interactive Tkinter-based application for missile subsystem data visualization.
- Implemented structured navigation to display categorized missile components dynamically.

2.Implemented Dynamic File Selection and Processing

- Enabled users to load CSV and Excel files dynamically using file dialogs.
- Ensured compatibility with various datasets by integrating Pandas for data handling.

3.Enhanced Data Visualization with Interactive Plots

- Used Plotly Express to generate interactive line charts with multiple Y-axis selection.
- Provided customized legends and axis labels for better data interpretation.

4.Integrated Image Processing for Improved UI

- Successfully displayed DRDO and missile-related images in the main interface.
- Improved user engagement with a well-structured and visually appealing GUI.

5.Implemented Robust Error Handling and Validation

- Incorporated try-except blocks to handle file errors, empty datasets, and invalid selections.
- Displayed meaningful error messages to improve user experience and avoid crashes.

6.Scalable and Modular Codebase

- Designed the system to allow future expansion by adding more missile subsystems.
- Used modular functions for easier maintenance and code reusability.

5.2 Contributions:

1.Improved Accessibility for Engineers and Researchers

- Provided an easy-to-use interface for analyzing missile component data.
- Enabled quick selection and visualization of different subsystems.

2.Optimized User Experience with Structured UI

- Used color-coded buttons, dropdowns, and scrollable checkboxes for smooth interaction.
- Placed "Back" buttons strategically for easy navigation between windows.

3.Enhanced Data Interpretation with Multi-Y Axis Selection

- Allowed users to select multiple Y-axis values dynamically for more detailed analysis.
- Improved decision-making with clear, interactive graphs.

4.Ensured Cross-Platform Compatibility

- Developed the application to run on Windows, macOS, and Linux without major modifications.
- Used Tkinter and Pandas, which are platform-independent libraries.

5.Reduced Manual Work in Data Exploration

- Automated the file selection, column detection, and plotting process, reducing manual effort.
- Eliminated the need for users to manually enter column names or load data separately.

6.Methodology

6.1 Install Python:

Download Python

- Go to the official Python website: https://www.python.org/downloads/
- Click **"Download Python 3.x.x"** (latest stable version).

Run the Installer

- Open the downloaded **.exe** file.
- **Check the box "Add Python to PATH"** before proceeding
- Click **Install Now**

Check Installation

- Type python –version in command prompt.

6.2 Install Libraries:

1.Tkinter

- Installation: Pre-installed with Python
- Purpose: Used for building the GUI (buttons, labels, windows).

2.Pandas

- Installation: pip install pandas
- Purpose: Used for reading and processing CSV and Excel files.

3.Plotly

- Installation: pip install plotly
- Purpose: Creates interactive plots for data visualization.

4.Pillow

- Installation: pip install pillow
- Purpose: Handles image loading and processing in Tkinter.

7.CODE

```
import tkinter as tk
from tkinter import messagebox
from tkinter import filedialog as fd, StringVar, IntVar, Menu
import pandas as pd
import plotly.express as px
from PIL import Image, ImageTk

static_data = {
    'Astra mk-II': ['OBC', 'RPF', 'DLT', 'INS', 'SAM', 'SEEKER'],
    'Astra LCA': ['OBC', 'RPF', 'DLT', 'INS', 'SAM', 'SEEKER'],
    'Astra VL': ['OBC', 'RPF', 'DLT', 'INS', 'SAM', 'SEEKER']
}
def show_data(item):
    new_window = tk.Toplevel(root)
    new_window.title(f"Details of {item}")
    new_window_width = 600
    new_window_height = 500
    screen_width = root.winfo_screenwidth()
    screen_height = root.winfo_screenheight()
    x_position = (screen_width // 2) - (new_window_width // 2)
    y_position = (screen_height // 2) - (new_window_height // 2)

    new_window.geometry(f"{new_window_width}x{new_window_height}+{x_position}+{y_position}")

    associated_items = static_data.get(item, [])
    button_color = ['#ffc0cb', '#ffc0cb', '#ffc0cb', '#ffc0cb', '#ffc0cb', '#ffc0cb']
    for idx, sub_item in enumerate(associated_items):
        color = button_color[idx % len(button_color)]
        btn = tk.Button(
            new_window,
            text=sub_item,
            command=lambda s=sub_item: open_file_and_setup_columns(s),
            bg=color,
            fg='black',
            width=25,
            height=2
        )
        btn.pack(padx=10, pady=5)
    back_button = tk.Button(
        new_window,
        text="Back",
        command=new_window.destroy,
        bg="grey",
        fg="black",
        width=10,
            height=2
        )
        back_button.place(x=10, y=10)
    def select_file():
        file_path = fd.askopenfilename(
            title="Select a Data File",
            filetypes=[("CSV Files", ".csv"), ("Excel Files", ".xlsx")]
        )
        return file_path
    def open_file_and_setup_columns(button_text):
        file_path = select_file()
        if file_path:
            try:
                if file_path.endswith('.csv'):
                    df = pd.read_csv(file_path)
                elif file_path.endswith('.xlsx'):
                    df = pd.read_excel(file_path)
                else:
                    raise ValueError("Unsupported file type selected.")

                if df.empty:
                    raise ValueError("The selected file is empty.")
            except Exception as e:
                messagebox.showerror("Error", f"File Error: {e}")
                return

            # Create a new window for plotting
            plot_window = tk.Toplevel(root)
            plot_window.title(f"Plot Data for {button_text}")
            plot_window_width = 600
            plot_window_height = 500
            screen_width = root.winfo_screenwidth()
            screen_height = root.winfo_screenheight()
            x_position = (screen_width // 2) - (plot_window_width // 2)
            y_position = (screen_height // 2) - (plot_window_height // 2)
            plot_window.geometry(f"{plot_window_width}x{plot_window_height}+{x_position}+{y_position}")

            # Add Back button in the top-left corner
            back_btn = tk.Button(
                plot_window,
                text="Back",
                command=plot_window.destroy,  # Closes the plot window
                bg="grey",
                fg="black",
                width=10,
                height=2
            )
            back_btn.place(x=10, y=10)
```

```
    # Column selection dropdowns and options
    columns = df.columns.tolist()
    x_axis_var = StringVar()
    x_axis_var.set(columns[0])

    tk.Label(plot_window, text=f"Plotting window for {button_text}", fg='black').pack(pady=10)
    tk.Label(plot_window, text="Select X Axis:", bg='#ff00ff', fg='white').pack(pady=5)
    x_dropdown = tk.OptionMenu(plot_window, x_axis_var, *columns)
    x_dropdown.pack()

    # Y-axis checkbox section
    tk.Label(plot_window, text="Select Y Axis (multiple):", bg='#ff00ff', fg='white').pack(pady=5)
    selected_y_axes = {}
    dropdown_frame = tk.Frame(plot_window, relief=tk.RAISED, borderwidth=2)
    dropdown_frame.pack(pady=10)

    dropdown_canvas = tk.Canvas(dropdown_frame, width=250, height=200)
    scrollbar = tk.Scrollbar(dropdown_frame, orient="vertical", command=dropdown_canvas.yview)
    checkbutton_frame = tk.Frame(dropdown_canvas)

    checkbutton_frame.bind(
        "<Configure>",
        lambda e: dropdown_canvas.configure(scrollregion=dropdown_canvas.bbox("all"))
    )

    dropdown_canvas.create_window((0, 0), window=checkbutton_frame, anchor="nw")
    dropdown_canvas.configure(yscrollcommand=scrollbar.set)

    for col in columns:
        selected_y_axes[col] = IntVar()
        tk.Checkbutton(checkbutton_frame, text=col, variable=selected_y_axes[col]).pack(anchor='w', padx=5, pady=2)

    dropdown_canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
    scrollbar.pack(side=tk.RIGHT, fill=tk.Y)



        # Plot button
        plot_button = tk.Button(
            plot_window,
            text="Plot Data",
            command=lambda: plot_data_with_checkboxes(df, x_axis_var.get(), selected_y_axes, button_text),
            bg="#808080",
            fg="white",
            width=25,
            height=2
        )
        plot_button.pack(pady=20)
    else:
        print("No file selected")


def plot_data_with_checkboxes(df, x_axis, selected_y_axes, title):
    try:
        y_axes = [col for col, var in selected_y_axes.items() if var.get() == 1]
        if not y_axes:
            messagebox.showerror("Error", "Please select at least one Y-axis.")
            return
        fig = px.line(
            df,
            x=x_axis,
            y=y_axes,
            title=f"plot for {title}"
        )
        for idx, y_col in enumerate(y_axes):
            fig.data[idx].name = y_col

        fig.update_layout(
            showlegend=True,
            legend=dict(
                title="words",
                x=1.0, y=1.0,
                xanchor="left",
                yanchor="top",
                font=dict(size=14),
                bgcolor="rgba(255,255,255,0.8)",
                bordercolor="black",
                borderwidth=3
            )
        )
        fig.show()

    except Exception as e:
        messagebox.showerror("Error", f"Could not plot data: {e}")
```

```
#main window
root = tk.Tk()
root.title("Select Data")
window_width =600
window_height =500
screen_width = root.winfo_screenwidth()
screen_height = root.winfo_screenheight()
x_position = (screen_width // 2) - (window_width // 2)
y_position = (screen_height // 2) - (window_height // 2)

root.geometry(f"{window_width}x{window_height}+{x_position}+{y_position}")

#main window title
heading =tk.Label(root,text="Defence Research and Development Laboratory",font=("Arial",15,"bold"))
heading.pack(pady=20)

#load and display an image(drdo symbol)
# Load and display first image
image1 = Image.open(r"C:\Users\manid\Downloads\DRDO_Seal.png")
image1 = image1.resize((100, 100))  # Resize image if needed
photo1 = ImageTk.PhotoImage(image1)

# Load and display second image
image2 = Image.open(r"C:\Users\manid\Downloads\missile.png")  # Replace with second image
image2 = image2.resize((100, 100))
photo2 = ImageTk.PhotoImage(image2)

# Create a frame to hold the images
frame = tk.Frame(root)
frame.pack(pady=20)

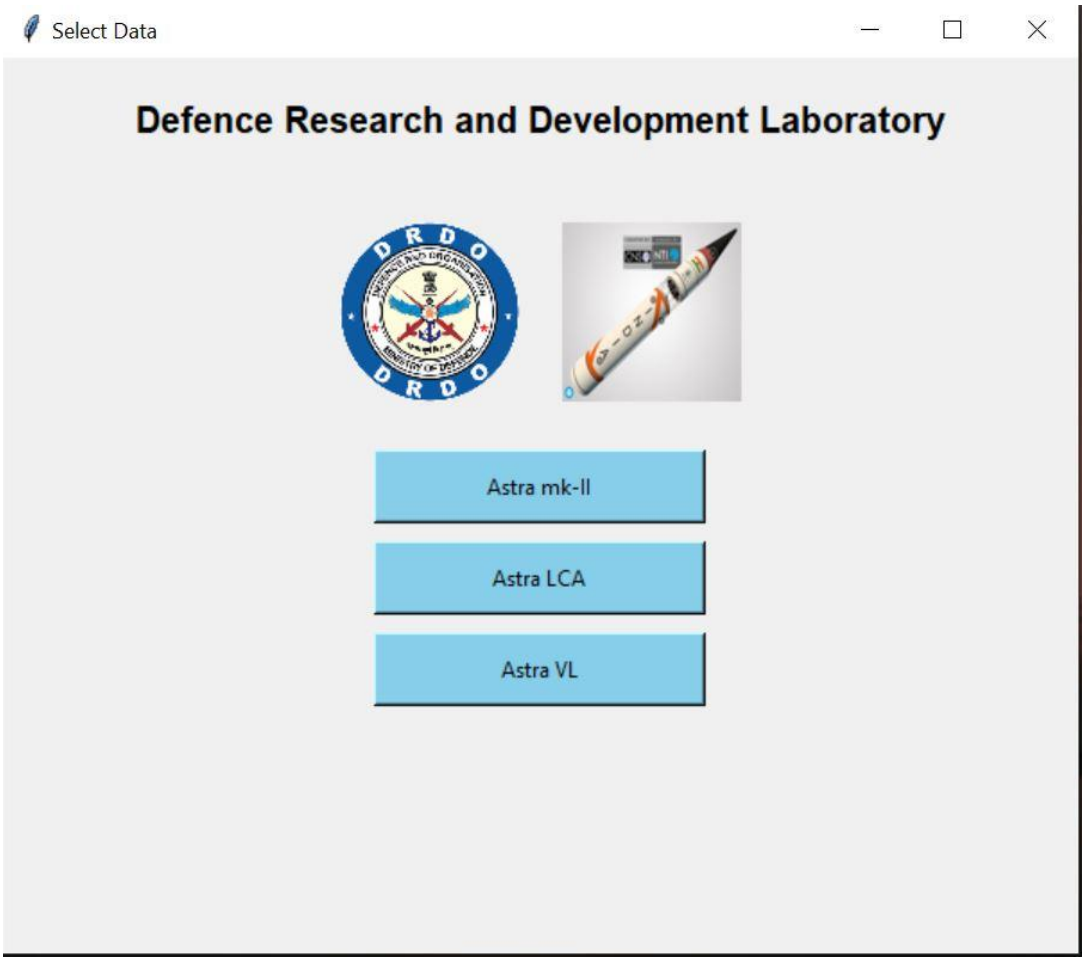
# Labels to display the images side by side
label1 = tk.Label(frame, image=photo1)
label1.pack(side="left", padx=10)  # Add spacing between images

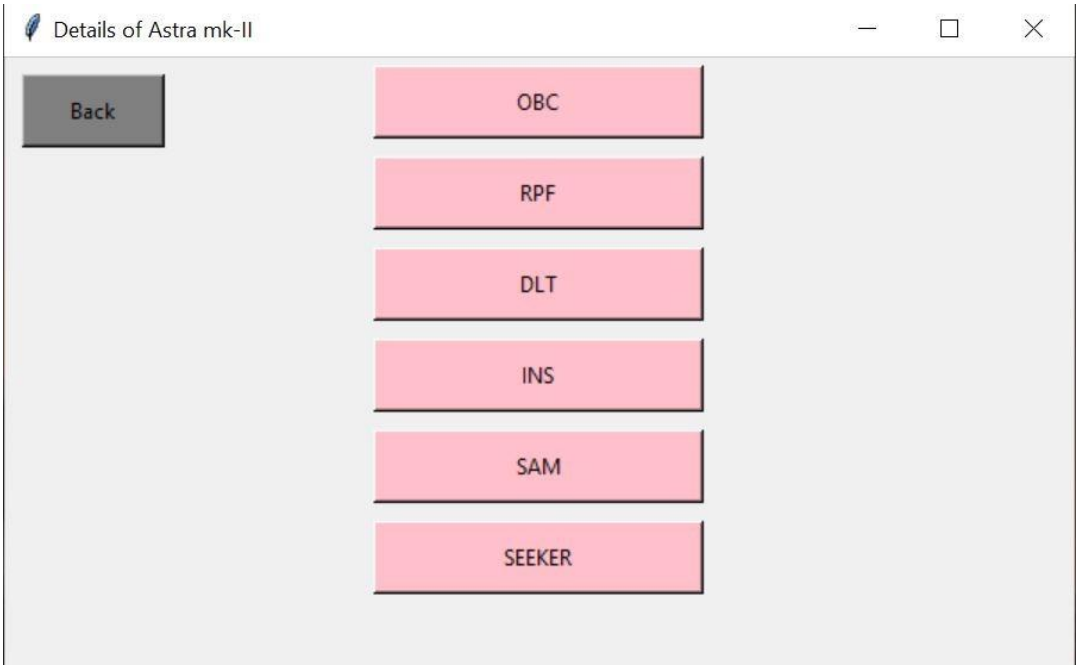
label2 = tk.Label(frame, image=photo2)
label2.pack(side="left", padx=10)

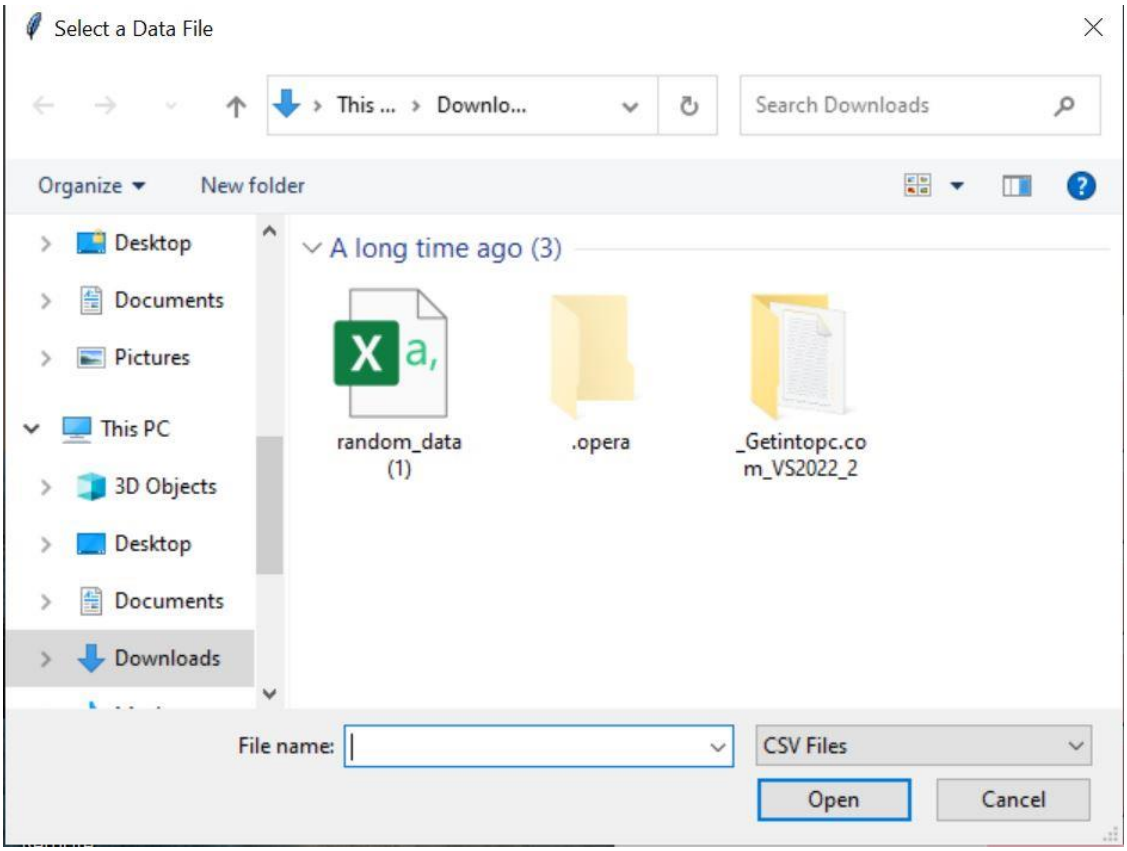

items = ['Astra mk-II', 'Astra LCA', 'Astra VL']
button_colors = ['#87ceeb', '#87ceeb', '#87ceeb']
for col,item in enumerate(items):
    tk.Button(
        root,
        text=item,
        command=lambda b=item: show_data(b),bg=button_colors[col],
        fg='black',width=25,height=2
    ).pack(padx=10, pady=5)

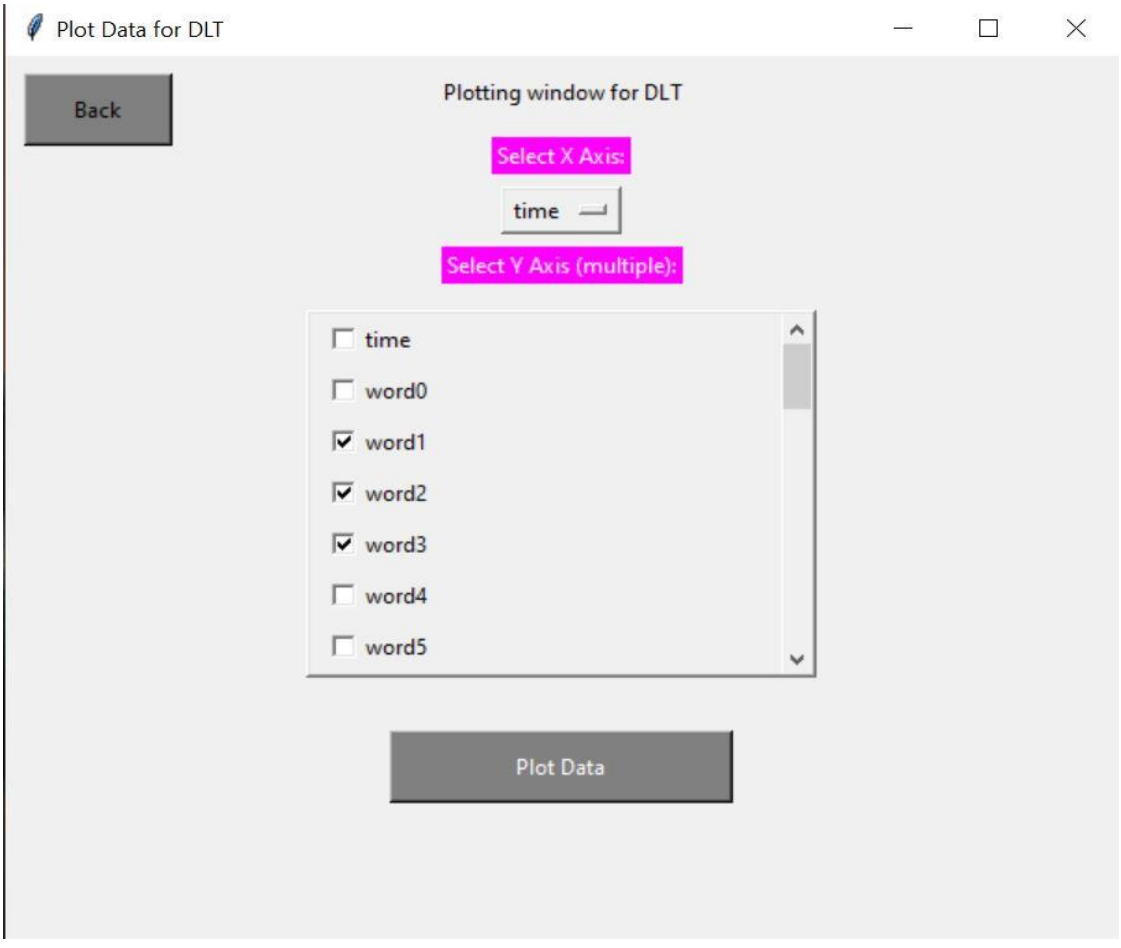
root.mainloop()
```

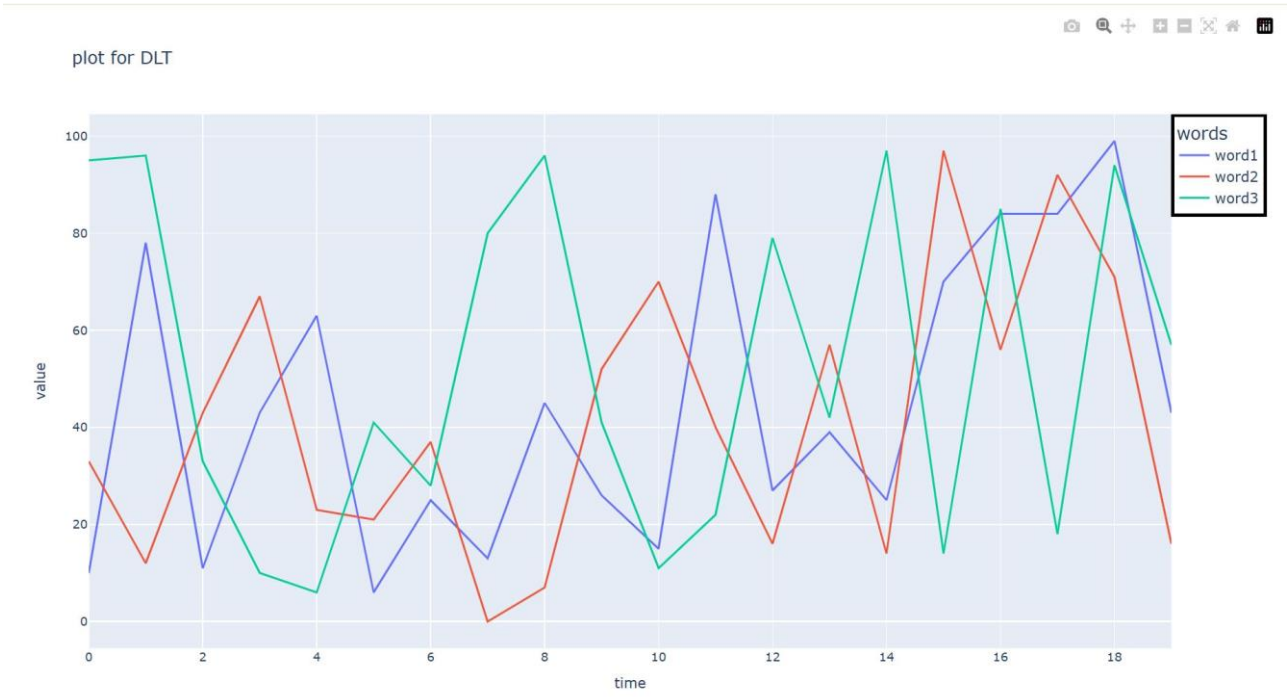
8.Result Screens






Select a Data File
This ...
Downlo...
Search Downloads
Organize
New folder
Desktop
Documents
Pictures
This PC
3D Objects
Desktop
Documents
Downloads
A long time ago (3)
random_data (1)
.opera
_Getintopc.com_VS2022_2
File name:
CSV Files
Open
Cancel


Plot Data for DLT
Back
Plotting window for DLT
Select X Axis:
time
Select Y Axis (multiple):
time
word0
word1
word2
word3
word4
word5
Plot Data


plot for DLT
words
word1
word2
word3
value
time
0
20
40
60
80
100
0
2
4
6
8
10
12
14
16
18

9.Conclusion

The development of this interactive GUI-based missile subsystem data visualization tool successfully integrates Tkinter, Pandas, Plotly, and Pillow to provide an efficient and user- friendly platform for analyzing test data.

By enabling users to select missile subsystems, load test datasets, and dynamically visualize key performance parameters, the tool enhances the ability of engineers and analysts to interpret large volumes of data effectively.

The integration of interactive data visualization using Plotly allows for detailed trend analysis, improving decision-making and performance evaluation. Additionally, the incorporation of image handling using Pillow ensures a visually appealing and professional interface.

Overall, this tool streamlines the process of missile subsystem performance assessment, making it more intuitive, efficient, and data driven. Future enhancements could include advanced statistical analysis, real-time data streaming, and additional visualization features to further improve analytical capabilities.